

Spacecoin Airdrop Contract Security Audit

: Spacecoin Airdrop Contract Security Audit

September 17, 2025

Revision 1.0

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Spacecoin Airdrop Contract Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	5
Scope	5
Code Revision	5
Severity Categories	6
Status Categories	8
Finding Breakdown by Severity	9
Findings	10
Summary	10
#1 SPACEAIRDROP-001 Incomplete signature binding in claimWithSignature() can allow stale claims and fee front-running	11
#2 SPACEAIRDROP-002 Excessive operator privileges can enable unauthorized reward manipulation	14
#3 SPACEAIRDROP-003 updateRewards() does not reject reclaimed campaigns	17
#4 SPACEAIRDROP-004 _updateRewards() does not validate the campaign token address	18
#5 SPACEAIRDROP-005 Minor Suggestions	19
Revision History	22

Executive Summary

From August 29, 2025, Theori's ChainLight team conducted a security audit of the Space airdrop contract over a one-week period. The assessment focused on reward allocation and claim flows, including campaign creation and updates, signature-based claiming, fee handling, reclaim logic, and operator/admin privilege boundaries. During the audit, several issues were identified that could affect reward integrity and access control, including stale signature usage after reward updates, incomplete signature binding that could allow fee front-running, overly broad operator privileges that could enable unauthorized reward manipulation, and missing validation in campaign update logic. In addition, several minor issues were identified relating to naming consistency, input validation, claim usability, signature revocation, event emission, and privileged role management.

Audit Overview

Scope

Name	Spacecoin Airdrop Contract Security Audit
Target / Version	Git Repository (<code>gluwa/Airdrop-Claim</code>): commit <code>0e0c3a4df96cddf315d2937e6318fef4ad58f2a3</code>
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	• N/A
High	0	• N/A
Medium	0	• N/A
Low	4	• SPACEAIRDROP-001 • SPACEAIRDROP-002 • SPACEAIRDROP-003 • SPACEAIRDROP-004
Informational	1	• SPACEAIRDROP-005
Note	0	• N/A

Findings

Summary

#	ID	Title	Severity	Status
1	SPACEAIRDROP-001	Incomplete signature binding in <code>claimWithSignature()</code> can allow stale claims and fee front-running	Low	Acknowledged
2	SPACEAIRDROP-002	Excessive operator privileges can enable unauthorized reward manipulation	Low	Acknowledged
3	SPACEAIRDROP-003	<code>updateRewards()</code> does not reject reclaimed campaigns	Low	Patched
4	SPACEAIRDROP-004	<code>_updateRewards()</code> does not validate the campaign token address	Low	Patched
5	SPACEAIRDROP-005	Minor Suggestions	Informational	Patched

#1 SPACEAIRDROP-001 Incomplete signature binding in

`claimWithSignature()` can allow stale claims and fee front-running

ID	Summary	Severity
SPACEAIRDROP-001	The signature used in <code>claimWithSignature()</code> is not bound to the claimant's current <code>totalReward</code> or to the intended fee recipient. As a result, a signature may remain valid even after <code>updateRewards()</code> changes the reward amount, and a third party may front-run the transaction and receive the fee.	Low

Description

`claimWithSignature()` allows any caller to submit a signed claim on behalf of `owner` :

```
bytes32 structHash = keccak256(  
    abi.encode(  
        CLAIM_TYPEHASH,  
        owner,  
        campaignNameHash,  
        fee,  
        _useNonce(owner),  
        deadline  
    )  
);
```

The signed data includes `owner`, `campaignNameHash`, `fee`, `nonce`, and `deadline`, but it does not include the claimant's current reward amount stored in `_campaigns[campaignNameHash].rewards[owner].totalReward`.

This becomes problematic because `updateRewards()` allows the operator to modify `totalReward` for an unclaimed account. If the frontend requests a signature based on a fee

derived from the reward amount, such as 1% of `totalReward`, that fee assumption may become outdated after `updateRewards()` changes the reward. However, the old signature may still remain valid because the signed payload does not commit to the reward value used when the signature was generated.

In addition, `_claim()` transfers the fee to `msg.sender` :

```
IERC20(c.token).transfer(user, r.totalReward - fee);  
IERC20(c.token).transfer(msg.sender, fee);
```

Because `claimWithSignature()` can be called by any address, a third party may front-run a signed claim transaction and execute it first. Since the signed payload does not bind the intended fee recipient, the front-runner can receive the fee instead of the party originally expected to submit the claim.

As a result, the current signature scheme does not fully bind all claim-sensitive parameters, allowing stale signatures to remain usable after reward updates and enabling fee interception through front-running.

Impact

Low

A previously issued signature may still be used after `totalReward` has changed, causing the claim to be executed under outdated fee assumptions. In addition, any third party that front-runs the signed transaction may receive the fee, resulting in unintended fee distribution.

Recommendation

The signed payload should include

`_campaigns[campaignNameHash].rewards[owner].totalReward` and the intended `feeRecipient`. This ensures that signatures become invalid when the reward amount changes and cannot be reused by another caller to redirect the fee.

Remediation

Acknowledged

The issue was acknowledged by the team. The team noted that the fee is intended to be set only at a level sufficient to cover transaction gas costs, and therefore considered signature front-running to be an acceptable limitation with limited practical impact.

#2 SPACEAIRDROP-002 Excessive operator privileges can enable unauthorized reward manipulation

ID	Summary	Severity
SPACEAIRDROP-002	The use of <code>unchecked</code> arithmetic in <code>_addRewards()</code> may allow a malicious operator to manipulate <code>c.totalAmount</code> through overflow and create underfunded campaigns. In addition, campaigns are not bound to the operator that created them, allowing any operator to update rewards for campaigns created by other operators. Together, these issues can enable a malicious operator to misuse campaign accounting and interfere with rewards belonging to other operators.	Low

Description

In `_addRewards()`, the contract updates `c.totalAmount` inside an `unchecked` block while iterating over `params.totalRewards`:

```
unchecked {  
    c.totalAmount += params.totalRewards[i];  
    ++i;  
}
```

Because overflow checks are disabled, a malicious operator may provide crafted `totalRewards` values that cause `c.totalAmount` to overflow. In the extreme case, `c.totalAmount` may become significantly smaller than the actual sum of assigned rewards, including potentially wrapping to zero, while reward entries are still recorded for selected accounts.

Since `_addRewards()` later transfers only `c.totalAmount` from the operator to the contract, this may allow a malicious operator to create an underfunded campaign without depositing the full amount corresponding to the configured rewards.

A related access control issue exists in `updateRewards()`. The `Campaign` struct does not store the operator that created the campaign, and `updateRewards()` is callable by any address with `OPERATOR_ROLE`. As a result, one operator may modify the rewards of a campaign created by another operator:

```
function updateRewards(...) external override onlyRole(OPERATOR_ROLE) {
    _updateRewards(...);
}
```

This is inconsistent with the expected privilege boundary of the operator role. Unlike `reclaim()`, which is restricted to the admin role, `updateRewards()` currently allows any operator to alter reward allocations across all campaigns, including campaigns they did not create. A malicious operator could therefore tamper with or redirect value from campaigns managed by other operators.

Impact

Low

A malicious operator may create underfunded campaigns by exploiting unchecked overflow in `_addRewards()`. In addition, any operator may modify campaigns created by other operators through `updateRewards()`. This breaks campaign isolation between operators and may allow unauthorized reward manipulation or theft of tokens associated with other operators' campaigns.

Recommendation

Remove the `unchecked` block in `_addRewards()` so that reward total calculations revert on overflow. In addition, store the campaign creator or operator address in the `Campaign` struct and restrict `updateRewards()` so that only the same operator, or the admin, can update that campaign.

Remediation

Acknowledged

The issue was acknowledged by the team. The team stated that the `OPERATOR_ROLE` will be managed securely through a multisig or hardware wallet, and therefore assumed that compromise of operator privileges or malicious operator behavior is highly unlikely.

#3 SPACEAIRDROP-003 `updateRewards()` does not reject

reclaimed campaigns

ID	Summary	Severity
SPACEAIRDROP-003	<code>_updateRewards()</code> can still be executed even when <code>c.reclaimed == true</code> . Since a reclaimed campaign has already ended and its remaining tokens have been withdrawn, allowing further reward updates is inconsistent with the expected campaign lifecycle.	Low

Description

In `_updateRewards()`, the function does not check whether the target campaign has already been reclaimed. If `c.reclaimed == true`, the campaign has already ended and the remaining tokens have been recovered through `reclaim()`. However, `updateRewards()` may still modify reward data for that campaign.

Impact

Low

Reward data may be modified for campaigns that are already finalized and reclaimed, which can lead to inconsistent campaign state.

Recommendation

`_updateRewards()` should revert when `c.reclaimed == true`.

Remediation

Patched

commit: `ea195b7472fb4063b9ee74bc794bfee7b2a86615`

#4 SPACEAIRDROP-004 `_updateRewards()` does not validate the campaign token address

ID	Summary	Severity
SPACEAIRDROP-004	<code>_updateRewards()</code> does not verify that <code>params.token</code> matches the existing campaign token, <code>c.token</code> . As a result, if the two addresses differ, the function may pull additional funds using <code>params.token</code> instead of the actual campaign token when increasing the reward allocation.	Low

Description

In `_updateRewards()`, the function accepts `params.token` but does not check whether it matches `c.token` for the target campaign. If the reward allocation is increased, the contract transfers the additional amount from `msg.sender` using `params.token`. Therefore, when `params.token != c.token`, the contract may fund the campaign with an unintended token even though the campaign itself is associated with `c.token`.

Impact

Low

The contract may pull the wrong token when topping up a campaign, resulting in inconsistent campaign funding and incorrect token accounting.

Recommendation

`_updateRewards()` should verify that `params.token == c.token` and revert otherwise.

Remediation

Patched

commit: `26947e6e6388b6ced2dc3be7b2a5227669728c5c`

#5 SPACEAIRDROP-005 Minor Suggestions

ID	Summary	Severity
SPACEAIRDROP-005	The description includes multiple suggestions for preventing incorrect settings caused by operational mistakes, mitigating potential issues, and improving code maturity and readability.	Informational

Description

1. It is recommended to rename the `RedeemableAirdrop.status` variable to `requiredAdditionalVerification`, matching the variable name used in the `Reward` struct.
2. It is recommended to execute `IERC20(c.token).transfer(msg.sender, fee);` in `_claim()` only when the `fee` is greater than 0.
3. It is necessary to validate in `_addRewards()` that `params.startDate` is greater than `block.timestamp`.
4. The `claimByToken()` function causes the transaction to fail if the user has already claimed from even one of the `_campaignsList[]`. This negatively impacts user experience. It is recommended to modify the logic so that if `_campaignsList[i]` has already been claimed, it is skipped instead of reverting, while continuing to process the remaining claims.
5. A feature to revoke `claimWithSignature` signatures should be added. Therefore, it is recommended to implement a wrapper function that can call `_useNonce(msg.sender)`.
6. It is recommended to add an event to the `updateClaimantStatus()` function.
7. It is recommended that the address with `DEFAULT_ADMIN_ROLE` privileges be managed through a multisig.

Impact

Informational

Recommendation

Consider applying the suggestions in the description above.

Remediation

Patched

All issues, except for Minor Issue 5, were patched as recommended.

Revision History

Version	Date	Description
1.0	September 17, 2025	Initial version

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

